d) Forward Chaining

Aim:

To implement Forward Chaining (data-driven reasoning) using PROLOG.

Procedure:

1. Open Notepad (or the SWI-Prolog editor).
2. Write the facts and rules and save the file with **.pl** extension.
3. In SWI-Prolog, load the program using **File -> Consult** and select the file.
4. Type the query at the **?-** prompt, ending it with a full stop (.). Use **;** to get further answers.
5. Observe and record the output.

Prolog Code:

Exercise 1: Simple Inference

```
:- dynamic known/1.

% Initial knowledge
start :- assert(known(a)), assert(known(b)), infer.

% Rule
infer :- known(a), known(b), \+ known(c), assert(known(c)), write('Inferred c'), nl.

% Query
% ?- start.
```

Output:

Inferred c

Exercise 2: Fire Alert System

```
:- dynamic known/1.

% Rule to infer fire
check :- known(smoke), known(heat), assert(known(fire)), write('Fire Alert!'), nl.

% Start rule
start :- assert(known(smoke)), assert(known(heat)), check.

% Query
% ?- start.
```

Output:

Fire Alert!

Exercise 3: Water Purity Detection

```
:- dynamic known/1.

% Rule to infer pollution
infer_pollution :- known(cloudy_water), known(bad_smell), assert(known(polluted)),
                write('Water is polluted'), nl.

start :- assert(known(cloudy_water)), assert(known(bad_smell)), infer_pollution.

% Query
% ?- start.
```

Output:

Water is polluted

Result:

Thus Forward Chaining was implemented using PROLOG and executed successfully.